

UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

DEPARTMENT OF INTERNET OF THINGS AND ROBOTICS ENGINEERING (IRE)

FACULTY OF CYBER PHYSICAL SYSTEM ENGINEERING

LAB REPORT 01

SMS SPAM DETECTION USING NAIVE BAYES CLASSIFIER

COURSE CODE: AI 484

COURSE TITLE: ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING SESSIONAL

SUBMITTED BY:

Shahriar Alom Masud

Student ID: **2201032** | Session: 2022–2023

Department of Internet of Things and Robotics Engineering (IRE), UFTB

SUBMITTED TO:

Suman Saha

Assistant Professor

Department of Internet of Things and Robotics Engineering (IRE)

University of Frontier Technology, Bangladesh

OCTOBER 13, 2026

1. Problem Statement and Objectives

Short Message Service (SMS) spam is an ongoing security and user experience challenge in mobile telecommunications. The goal of this laboratory session is to build a machine learning pipeline that automatically identifies whether an incoming SMS is legitimate (*ham*) or spam, using the Naive Bayes classification algorithm.

Assigned Tasks:

1. Load and examine the SMS Spam Collection dataset.
2. Clean and preprocess message text (lowercase transformation, punctuation removal, tokenization).
3. Convert raw text into numerical features using Bag-of-Words vectorization.
4. Train a Multinomial Naive Bayes classifier on a stratified training split.
5. Test the model on unseen data and report standard evaluation metrics (Accuracy, Precision, Recall, and Confusion Matrix).
6. Predict the class for the specific test message:

   "Congratulations! You have won a free ticket. Call now!"

GitHub Repository Link:
https://github.com/Masud744/AI-ML-Lab/tree/main/Lab_01

2. Mathematical Background of Naive Bayes

The Naive Bayes classifier is built on Bayes' Theorem with the naive assumption of conditional independence among feature terms given the target class.

For an SMS message represented as a sequence of words $\mathbf{w} = (w_1, w_2, \ldots, w_n)$, the posterior probability of belonging to class $c \in \{\text{ham}, \text{spam}\}$ is expressed as:

$$P(c \mid \mathbf{w}) = \frac{P(c) \cdot P(\mathbf{w} \mid c)}{P(\mathbf{w})} \tag{1}$$

By assuming words occur independently given the class label:

$$P(\mathbf{w} \mid c) = \prod_{i=1}^{n} P(w_i \mid c) \tag{2}$$

The predicted class $\hat{c}$ is the one that yields the highest posterior probability:

$$\hat{c} = \arg \max_{c \in \{\text{ham,spam}\}} P(c) \prod_{i=1}^{n} P(w_i \mid c) \tag{3}$$

In practice, if a word from the test set never appeared in the training set for a particular class, $P(w_i \mid c)$ would evaluate to zero, making the entire product zero. To prevent this, **Laplace smoothing** ($\alpha = 1$) is used:

$$P(w_i \mid c) = \frac{N_{c,i} + \alpha}{N_c + \alpha \cdot |V|} \tag{4}$$

where $N_{c,i}$ is the count of word w_i in class c, N_c is the total count of words in class c, and $|V|$ is the total number of unique words across the training corpus.

3. Methodology and Preprocessing Pipeline

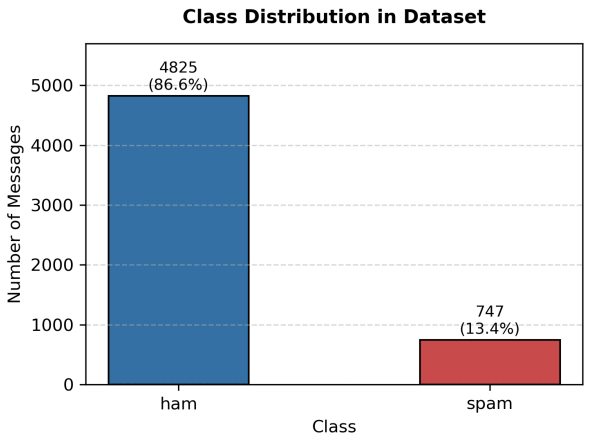


Figure 1: SMS Class Distribution ($N = 5,572$)

Dataset Overview:
The experiment uses the SMS Spam Collection dataset, containing $5,572$ labeled text messages. As typical in real-world spam detection datasets, the classes are imbalanced:

- **Ham (Legitimate):** $4,825$ messages (86.6%)
- **Spam (Unwanted):** 747 messages (13.4%)

Text Preprocessing Steps:

1. **Case Folding:** All characters were converted to lowercase so that words like `Free`, `FREE`, and `free` are treated as identical tokens.
2. **Punctuation and Digit Removal:** Special symbols, punctuation marks, and digits were stripped to reduce vocabulary noise.
3. **Whitespace Cleanup:** Consecutive whitespace characters were condensed into single spaces.

Feature Extraction and Model Fitting:

- The preprocessed messages were partitioned using an **80-20 stratified split** ($4,457$ training records and $1,115$ testing records), preserving the class distribution across both sets.
- `CountVectorizer` was fit on the training split to produce Bag-of-Words feature vectors, discovering a vocabulary of $6,830$ unique words.
- A `MultinomialNB(alpha=1.0)` model was trained on the vectorized representations.

4. Experimental Results and Analysis

The trained classifier was evaluated on the 1,115 holdout test messages (966 ham, 149 spam). The overall performance metrics are summarized below:

Class	Precision	Recall	F1-Score	Support
Ham (Legitimate)	0.99	1.00	0.99	966
Spam	0.98	0.91	0.94	149
Overall Accuracy		**98.48%**		

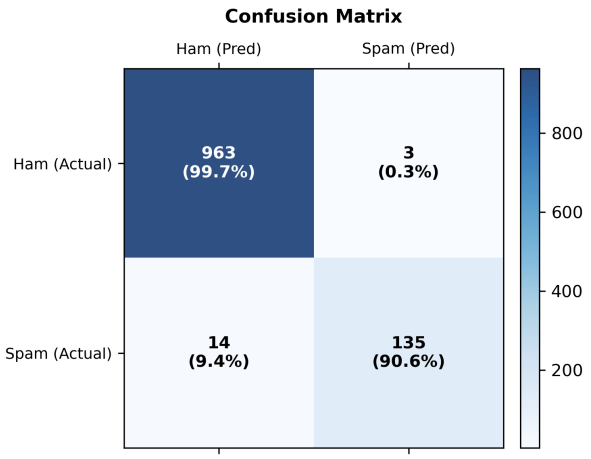


Figure 2: Confusion Matrix on Test Split

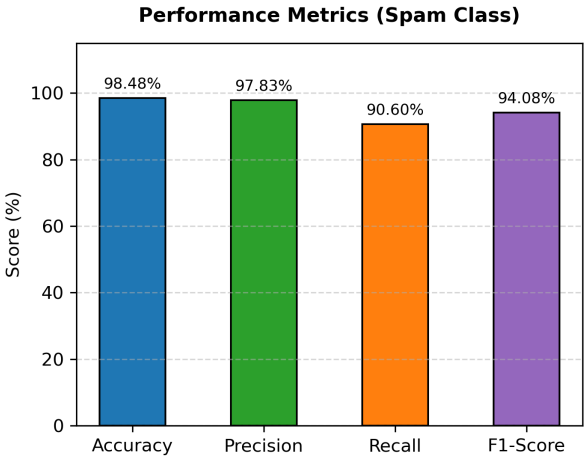


Figure 3: Performance Metrics Summary

Analysis of Results:

- **False Positives:** Out of 966 legitimate messages in the test set, only 3 messages (0.3%) were mistakenly classified as spam. This yields a spam precision of 97.83%, ensuring legitimate communications are rarely blocked.
- **False Negatives:** Out of 149 actual spam messages, 14 were misclassified as ham (giving a recall of 90.60%). These typically consist of very brief messages or texts without standard spam promotion terms.

5. Evaluation on Assigned Test Query

As specified in the lab problem, the model was tested on the following incoming text:

"Congratulations! You have won a free ticket. Call now!"

Prediction Pipeline Details:

1. **Input Preprocessing:** The message was cleaned to:

   `"congratulations you have won a free ticket call now"`

2. **Vocabulary Mapping:** The cleaned tokens were converted into term frequency counts. Words like `"congratulations"`, `"won"`, `"free"`, `"ticket"`, `"call"`, and `"now"` have very high likelihood in spam messages within the training set.

3. **Prediction:** The model predicted the label **SPAM**.

4. **Probability Scores:**

   - $P(\text{Spam} \mid \text{Message}) = \mathbf{99.89}\%$
   - $P(\text{Ham} \mid \text{Message}) = \mathbf{0.11}\%$

Interpretation:
Because the message contains several strong advertising and incentive keywords that rarely appear in legitimate daily communications, the posterior probability for the spam class is near 100%. The classifier correctly identifies the message as spam.

6. Discussion and Conclusion

- **Efficiency:** Multinomial Naive Bayes is computationally lightweight. Preprocessing, vectorization, and training across more than $5,500$ messages completed in under one second.
- **Effectiveness:** Despite the assumption that words appear independently, Naive Bayes performs remarkably well for spam classification, achieving an overall accuracy of 98.48% on unseen data.
- **Summary:** All tasks outlined in the lab problem were completed successfully. The dataset was preprocessed, vectorized, and trained using Multinomial Naive Bayes, and the specific query was accurately classified as spam.

7. Python Source Code Listing (spam_classifier.py)

GitHub File Link:
https://github.com/Masud744/AI-ML-Lab/blob/main/Lab_01/src/spam_classifier.py

```python
import os
import re
import string
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score, precision_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB


def preprocess_text(text):
    # Convert text to lowercase
    text = text.lower()
    # Remove punctuation and digits
    text = re.sub(f"[{re.escape(string.punctuation)}0-9]", " ", text)
    # Remove multiple spaces
    text = re.sub(r"\s+", " ", text).strip()
    return text


def plot_visuals(df, y_test, y_pred, output_dir, metrics):
    os.makedirs(output_dir, exist_ok=True)
    acc, prec, rec, f1 = metrics

    # 1. Class Distribution
    plt.figure(figsize=(5, 3.8))
    counts = df["label"].value_counts()
    bars = plt.bar(counts.index, counts.values, color=["#3470a3", "#c94a4a"], width=0.45, edgecolor="black")
    plt.title("Class Distribution in Dataset", fontsize=11, fontweight="bold", pad=12)
    plt.xlabel("Class", fontsize=10)
    plt.ylabel("Number of Messages", fontsize=10)
    plt.ylim(0, 5700)
    plt.grid(axis="y", linestyle="--", alpha=0.5)

    for bar in bars:
        h = bar.get_height()
        pct = (h / len(df)) * 100
        plt.text(bar.get_x() + bar.get_width() / 2, h + 50, f"{h}\n({pct:.1f}%)", ha="center", va="bottom", fontsize=9)

    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, "class_distribution.png"), dpi=300)
    plt.close()

    # 2. Confusion Matrix
    cm = confusion_matrix(y_test, y_pred, labels=["ham", "spam"])
    fig, ax = plt.subplots(figsize=(4.8, 4))
    cax = ax.matshow(cm, cmap="Blues", alpha=0.85)

    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            val = cm[i, j]
            pct = (val / np.sum(cm[i, :])) * 100
            txt_color = "white" if val > cm.max() / 2 else "black"
            ax.text(j, i, f"{val}\n({pct:.1f}%)", ha="center", va="center", color=txt_color, fontsize=10, fontweight="bold")

    fig.colorbar(cax, fraction=0.046, pad=0.04)
    ax.set_xticks([0, 1])
    ax.set_yticks([0, 1])
    ax.set_xticklabels(["Ham (Pred)", "Spam (Pred)"], fontsize=9)
    ax.set_yticklabels(["Ham (Actual)", "Spam (Actual)"], fontsize=9)
    ax.set_title("Confusion Matrix", fontsize=11, fontweight="bold", pad=12)
    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, "confusion_matrix.png"), dpi=300, bbox_inches="tight")
    plt.close()

    # 3. Performance Metrics Bar Chart
    plt.figure(figsize=(4.8, 3.8))
    names = ["Accuracy", "Precision", "Recall", "F1-Score"]
    vals = [acc * 100, prec * 100, rec * 100, f1 * 100]
    bars = plt.bar(names, vals, color=["#1f77b4", "#2ca02c", "#ff7f0e", "#9467bd"], width=0.5, edgecolor="black")
    plt.title("Performance Metrics (Spam Class)", fontsize=11, fontweight="bold", pad=12)
    plt.ylabel("Score (%)", fontsize=10)
    plt.ylim(0, 115)
    plt.grid(axis="y", linestyle="--", alpha=0.5)
    for bar in bars:
        h = bar.get_height()
```

```
        plt.text(bar.get_x() + bar.get_width() / 2, h + 2, f"{h:.2f}%", ha="center", va="bottom",
            fontsize=8.5)
    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, "metrics_chart.png"), dpi=300, bbox_inches="tight")
    plt.close()


def main():
    base_dir = os.path.dirname(os.path.abspath(__file__))
    project_root = os.path.abspath(os.path.join(base_dir, ".."))
    data_path = os.path.join(project_root, "dataset", "SMSSpamCollection.txt")
    output_dir = os.path.join(project_root, "submission_report", "screenshots")

    # Load dataset
    df = pd.read_csv(data_path, sep="\t", header=None, names=["label", "message"])
    print(f"Loaded dataset: {len(df)} messages ({df['label'].value_counts()['ham']} ham, {df['label
        '].value_counts()['spam']} spam)")

    # Preprocess text
    df["clean_message"] = df["message"].apply(preprocess_text)

    # Train-test split (80% train, 20% test)
    X_train, X_test, y_train, y_test = train_test_split(
        df["clean_message"], df["label"], test_size=0.20, random_state=42, stratify=df["label"]
    )

    # Text vectorization using Bag of Words
    vectorizer = CountVectorizer()
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)
    print(f"Vocabulary size: {len(vectorizer.vocabulary_)} words")

    # Train Naive Bayes model
    model = MultinomialNB(alpha=1.0)
    model.fit(X_train_vec, y_train)

    # Evaluate on test set
    y_pred = model.predict(X_test_vec)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, pos_label="spam")
    rec = recall_score(y_test, y_pred, pos_label="spam")
    f1 = f1_score(y_test, y_pred, pos_label="spam")

    print(f"\n--- Model Performance ---\nAccuracy  : {acc * 100:.2f}%\nPrecision : {prec * 100:.2f}%
        (Spam)\nRecall    : {rec * 100:.2f}% (Spam)\nF1-Score  : {f1 * 100:.2f}% (Spam)")
    print(f"\nClassification Report:\n{classification_report(y_test, y_pred, target_names=['ham', '
        spam'])}")

    # Test sample message
    test_message = "Congratulations! You have won a free ticket. Call now!"
    cleaned_test = preprocess_text(test_message)
    test_vec = vectorizer.transform([cleaned_test])
    prediction = model.predict(test_vec)[0]
    probabilities = model.predict_proba(test_vec)[0]
    spam_idx = list(model.classes_).index("spam")
    ham_idx = list(model.classes_).index("ham")

    print(f"--- Test Query Prediction ---\nInput text : {test_message}\nPrediction : {prediction.
        upper()}\nProbability: Ham = {probabilities[ham_idx] * 100:.2f}%, Spam = {probabilities[
        spam_idx] * 100:.2f}%")

    # Save visual plots for report
    plot_visuals(df, y_test, y_pred, output_dir, (acc, prec, rec, f1))
    print("\nPlots saved to screenshots directory successfully.")


if __name__ == "__main__":
    main()
```